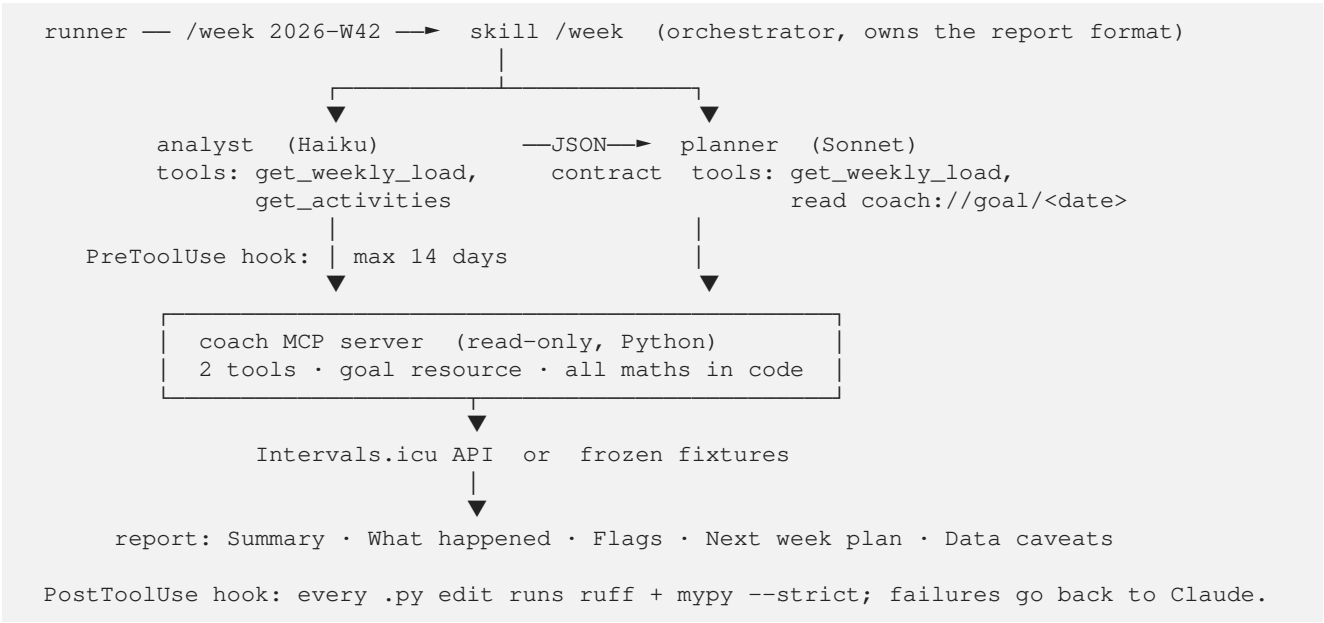
coach: a minimal agent that reviews a runner's week

Miguel · November 2026 · learning project on agent architecture with Claude Code

What I built

The problem. I have built large MCP servers before and watched agents get lost choosing between dozens of overlapping tools, re-adding numbers by hand and filling gaps with guesses. This project asks the opposite question: what is the *smallest* system that can still answer "how was my week?" and "what should next week look like?", reliably, from real training data?

The architecture. One command, two subagents, one read-only MCP server, two hooks.



The **analyst** reads one week and returns a fixed JSON object (distance, runs, % change, zero-week streak, data completeness, flags, next week's km ceiling). Every number, flag and ceiling comes from the server; the only model-made flag, `race_detected`, is marked as an inference. The **planner** receives that JSON in its prompt, reads the target race counted from the planned week, and proposes seven days under the ceiling. The **skill** chains them and writes the report. Each subagent sees only the tools it needs, in its own context window.

A real edge case: Strava and the Bilbao half marathon. My data lives in Intervals.icu, which cannot re-expose Strava-synced activities through its API. My most important run of the autumn, the Bilbao Night Half Marathon (22 km), came back as an empty stub. A naive system would read that week as rest. Here the server reports such runs as *blocked*, marks the week `data_complete: false`, withholds the % change and never counts it as a zero week. After importing the Strava archive, the agent sees the real race: it flags `race_detected` and `load_spike` (+481 % vs the previous weeks) and plans a recovery week. Three empty weeks later it flags a `layoff` and asks *why* I stopped before adding any load.

Quality.

- `mypy --strict` and `ruff` on all code; Pydantic V2 models with units in every field name.
- 15 offline tests on frozen fixtures: no network, no API key, reproducible.
- 4 headless evals (`claude -p "/week <week>"`): three real edge cases (low irregular volume, race plus spike, three-week layoff) and one clearly labelled synthetic build week. A script checks deterministic markers; a human review checks tone and plan quality. All 4 pass. The evals earned their keep: they caught the planner getting a different km ceiling than the analyst because the trend window depended on the caller's `n_weeks`. Fixed in code, with a regression test.
- Hooks tested with passing and failing cases, both standalone and inside a live session.

Principles applied

Principle	In one sentence	Where
Few tools	Two tools with distinct jobs, so the agent never has to choose between near-duplicates.	`src/coach/server.py`
Tools vs resources	The race goal is static data, so it is a resource the agent reads, not a tool it calls.	`server.py` (`coach://goal`)
Arithmetic in code	Totals, % change, streaks, flags and the next-week ceiling are computed and tested in Python; the model only interprets.	`src/coach/analytics.py`
Actionable errors	Every error says how to fix the call ("range is 152 days; max is 31, narrow it").	`server.py`, `.claude/hooks/`
Read-only by design	The server only issues GET requests and annotates every tool as read-only.	`server.py`, `sources.py`
Least privilege per subagent	Each subagent has an explicit tool allowlist; without it, it would inherit every tool.	`.claude/agents/*.md`
Right model for the job	Extraction runs on a small, fast model; planning on a stronger one.	`.claude/agents/*.md`
Context isolation	Analysis and planning run in separate context windows, so raw tool output never floods the planner.	`.claude/skills/week/SKILL.md`
Explicit hand-off contract	Subagents share no memory; the analyst returns a fixed JSON that the orchestrator passes on verbatim.	`analyst.md`, `SKILL.md`
Inference is labelled	What the model guesses (a race, from the run name) travels apart from what code computed.	`analyst.md` (`inferred_flags`)
Progressive disclosure	The report format lives in the skill and loads only when the report command runs, not in every session.	`.claude/skills/week/SKILL.md`
Hooks as deterministic guarantees	A rule that must always hold is code that runs on every call, not a sentence in a prompt.	`.claude/settings.json`, `.claude/hooks/`
Server validity vs client policy	The server rejects invalid requests (> 31 days); a stricter client hook (> 14 days) protects the context.	`limit_activity_range.py`
Secrets out of reach	A permission deny rule stops the agent from reading the API key file.	`.claude/settings.json`
No silent failure on partial data	Unreadable activities are reported and flagged, never treated as rest days.	`sources.py`, `analytics.py`
Missing data stays missing	No heart-rate sensor means HR is null and flagged, never estimated.	`models.py`, `analyst.md`
Evals with real edge cases	The system is tested on the weeks where it is most likely to be wrong.	`evals/`
Scope discipline	Ideas that would widen the system go to a roadmap instead of into the code.	`ROADMAP.md`

What I would do next. Detect races in code instead of from run names; validate the analyst's JSON with a hook before the planner sees it; add an LLM-as-judge step with a rubric for each eval case.

Not medical or coaching advice. A learning project about agent architecture.